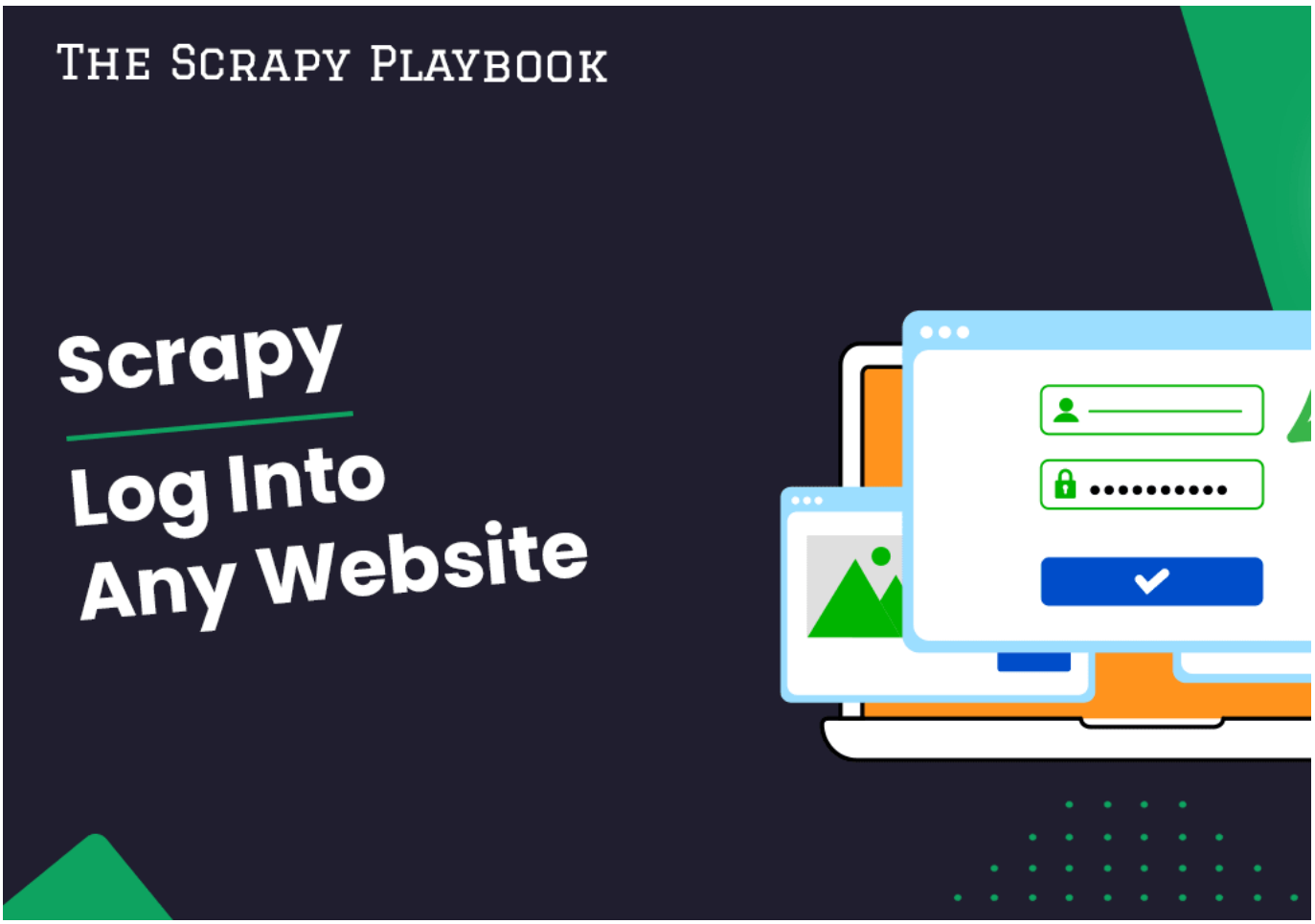


Python Scrapy Login Forms: How To Log Into Any Website

Logging into websites to scrape the data you need can be a tricky business.

You need to worry about filling out forms, managing headers, session/browser authenication, managing IP addresses.

Luckily for us Scrapy developers, Scrapy provides us a whole suite of tools and extensions we can use to log into any website.

In this guide we will look how the most popular methods to log into websites and other best practices:

- First Step: Analyse Login Process
- Login Method #1: Simple FormRequest
- Login Method #2: FormRequest With Hidden Data
- Login Method #3: Using Headless Browser To Login

- Not Getting Blocked After Logging In
- Risks Of Scraping Behind Logins

If you prefer to follow along with a video then check out the video tutorial version here:

Need help scraping the web?

Then check out **ScrapeOps**, the complete toolkit for web scraping.

Proxy Manager **Scraper Monitoring** **Job Scheduling**

First Step: Analyse Login Process

The first step in building a scraper that can login and scrape data behind a websites login page is understanding how their login process works.

To do this, first make sure you are logged out, then go to the Login page of the website you want to scrape.

Open the **Network Tab** of your **Developer Tools**, which we will use to analyze the network traffic and see how the websites login process works.

Then go through the login process in your browser.

Here you will want to look out for:

- The URL the website uses to login.
- The payload the login process sends.
- Any headers or cookies the login request needs.

After looking through the network tab your goal is to identify which login method will work best for your target website.

We will go through each of the 3 main login methods in detail below, however, here is a quick summary of when to use each method:

- **Login Method #1: Simple FormRequest -** Rare to see websites where this works anymore, however, if the website only requires you to send a username/email and password, and not security tokens then just send a simple FormRequest with the required data.
- **Login Method #2: FormRequest With Hidden Data -** Most websites require hidden data to log into a website that you must extract from the login page. If all the hidden data can be found on the login pages initial HTML response then you can extract them and add them to your FormRequest.
- **Login Method #3: Headless Browser Logins -** Some websites dynamically generate the hidden data needed to login in the browser or just have very complicated login processes. In cases like these we can login using a headless browser which will simplify our login process and then either continue scraping with the headless browser or pass the session cookies to our normal Scrapy requests.

Login Method #1: Simple FormRequest

At its simplest, logging into a website is just submiting data to a form.

Luckily for us, Scrapy makes it pretty easy to submit form data using Scrapy's inbuilt `FormRequest` class.

In this very simplistic example, we're going to use the `FormRequest` class to submit a login form that just takes the users email and password as inputs.

```
from scrapy import Spider
from scrapy.http import FormRequest


class SimpleLoginSpider(Spider):
    name = 'simple_login'

    def start_requests(self):
        login_url = 'http://example.com/login'
        return FormRequest(login_url,
                           formdata={'email': 'example@gmail.com',
'password': 'foobar'},
                           callback=self.start_scraping)

    def start_scraping(self, response):
        ## Insert code to start scraping pages once logged in
        pass
```

Here, we are simply configuring our scraper to `POST` our form data to the forms URL endpoint using the `FormRequest` class to log into the website, and once complete it will start scraping pages as defined in the `start_scraping()` method.

Scrapy will then handle the session cookies, etc. so that every page you request will be returned by the website as if you were logged in.

This is an overly simplistic example, as today very few websites just have simple forms for login pages. Most have some form of security feature that you need to factor in when making designing your scraper.

In the next section we will look at a more realistic example.

Login Method #2: FormRequest With Hidden

Data

Most signup and login forms these days use some form of hidden data to authenticate your request when you are logging into a website.

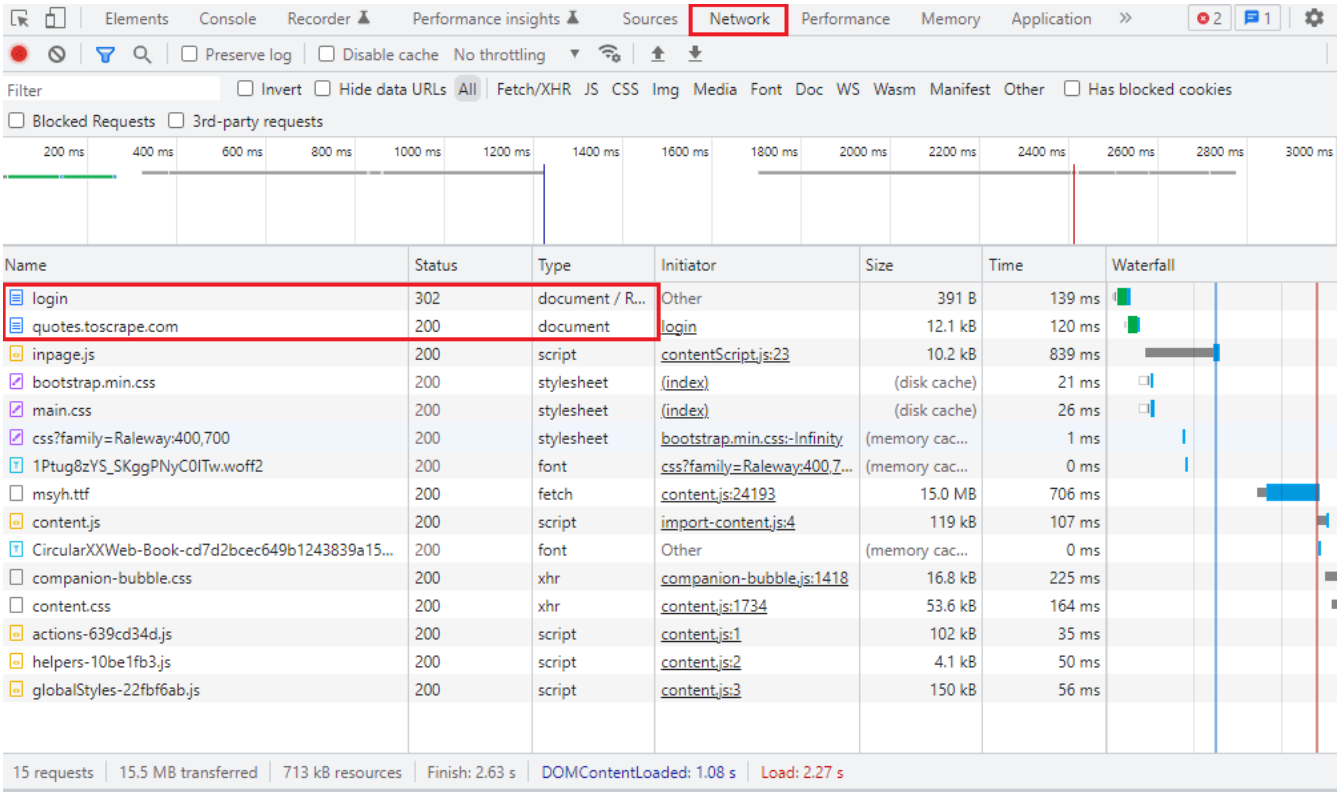
In cases like these, we first need to request the page with the login form itself and then extract this hidden data so we can then add it to our `FormRequest`.

For this example, we will use the QuotesToScrape login page, as the page won't change in the future so you can follow along with it.

First things first, we go through the login process in our browser with the **Network tab** of our **Developer Tools** open.

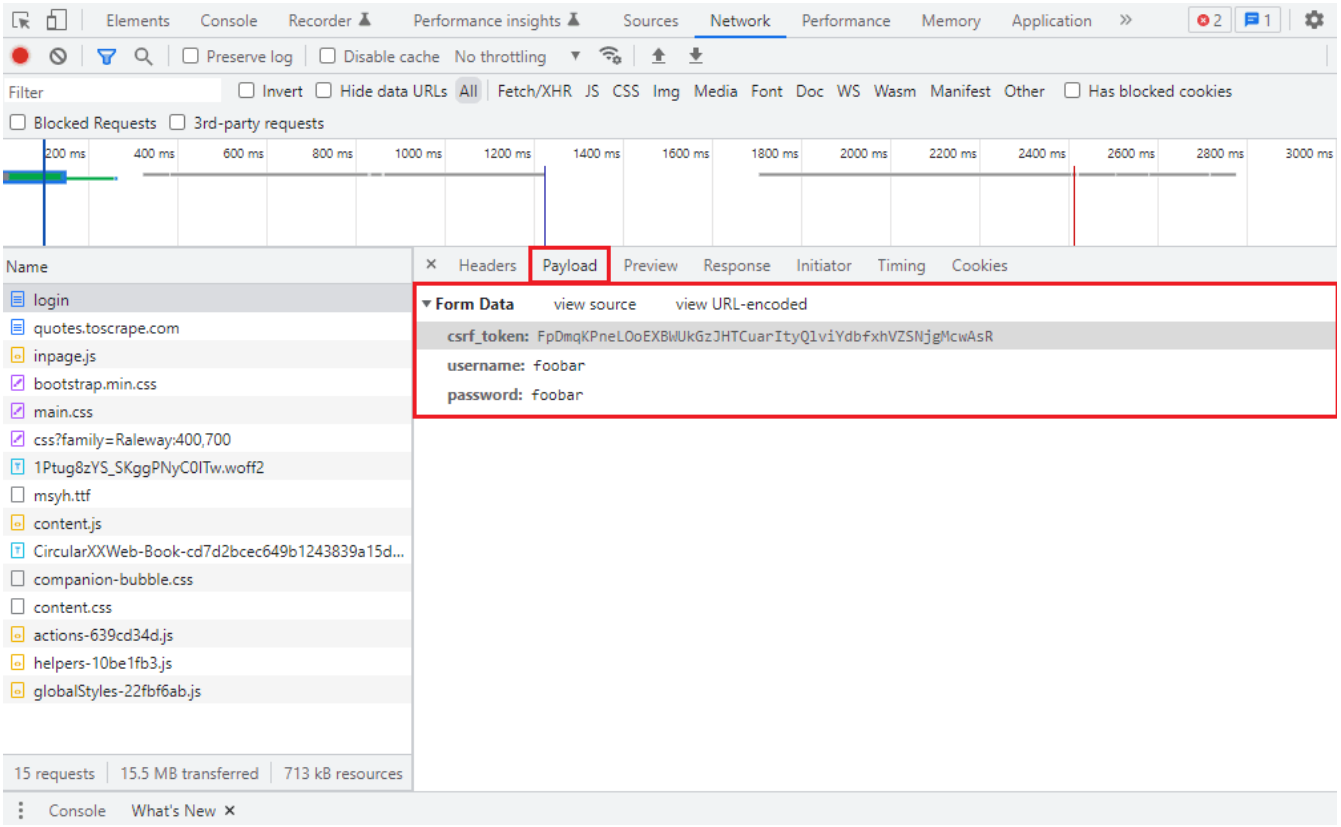
Whilst having the Network tab open and logged out, go to `http://quotes.toscrape.com/login` and enter `foobar` as both the username & password (anything works here). Then click login.

You should see the following network calls in your network tab. The two network calls we care about at the first two.



The first is a POST request that is used to send the form data to the server, which returns a 302 status code which means our request gets redirected to the main `quotes.toscrape.com` page after logging in.

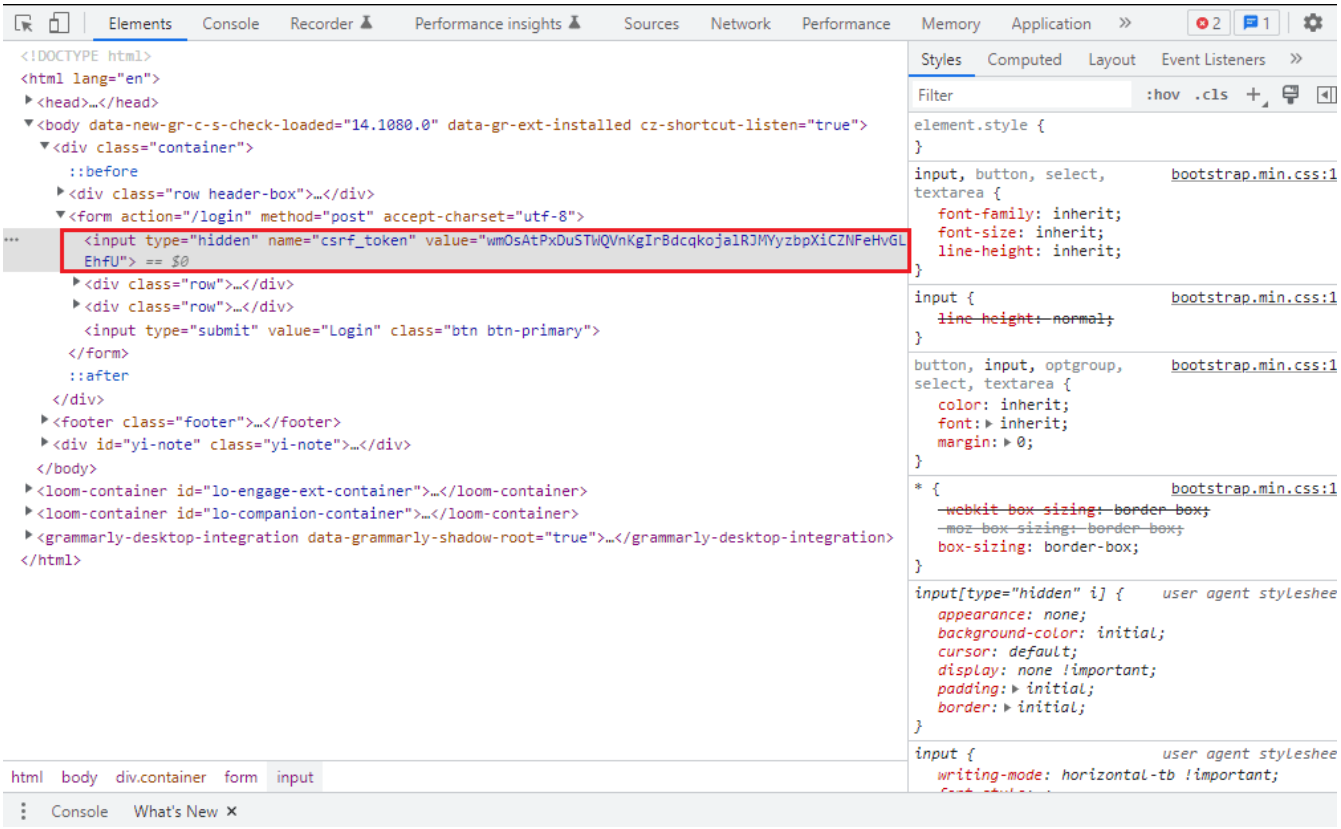
If we click on the **Payload** tab of the login POST request, we can see the data that is needed to successfully login.



Here, we can see the `username` and `password` that we entered into the boxes, however there is also a field called `csrf_token`.

> **CSRF TOKENS**
>
> A **CSRF token** is a unique, secret, unpredictable value that is generated by the server-side application and transmitted to the client so that it can be included in a subsequent HTTP request made by the client, like logging into to a website.

In this case, the `csrf_token` is generated by the **QuotesToScrape** server and added to the initial HTML response in a hidden `<input>` field.



Everytime we refresh the page the `csrf_token` will change, so we will need to first request the login page, extract the `csrf_token` from the hidden input field and then add that to our `FormRequest`.

```
import scrapy
from scrapy import Spider
from scrapy.http import FormRequest


class HiddenDataLoginSpider(Spider):
    name = 'hidden_data_login'

    def start_requests(self):
        login_url = 'http://quotes.toscrape.com/login'
        return scrapy.Request(login_url, callback=self.login)

    def login(self, response):
        token = response.css("form
input[name=csrf_token]::attr(value)").extract_first()
        return FormRequest.from_response(response,
                                         formdata={'csrf_token': token,
                                                   'password':
'foobar',
                                                   'username':
```

```
'foobar'},
                                        callback=self.start_scraping)

    def start_scraping(self, response):
        ## Insert code to start scraping pages once logged in
        pass
```

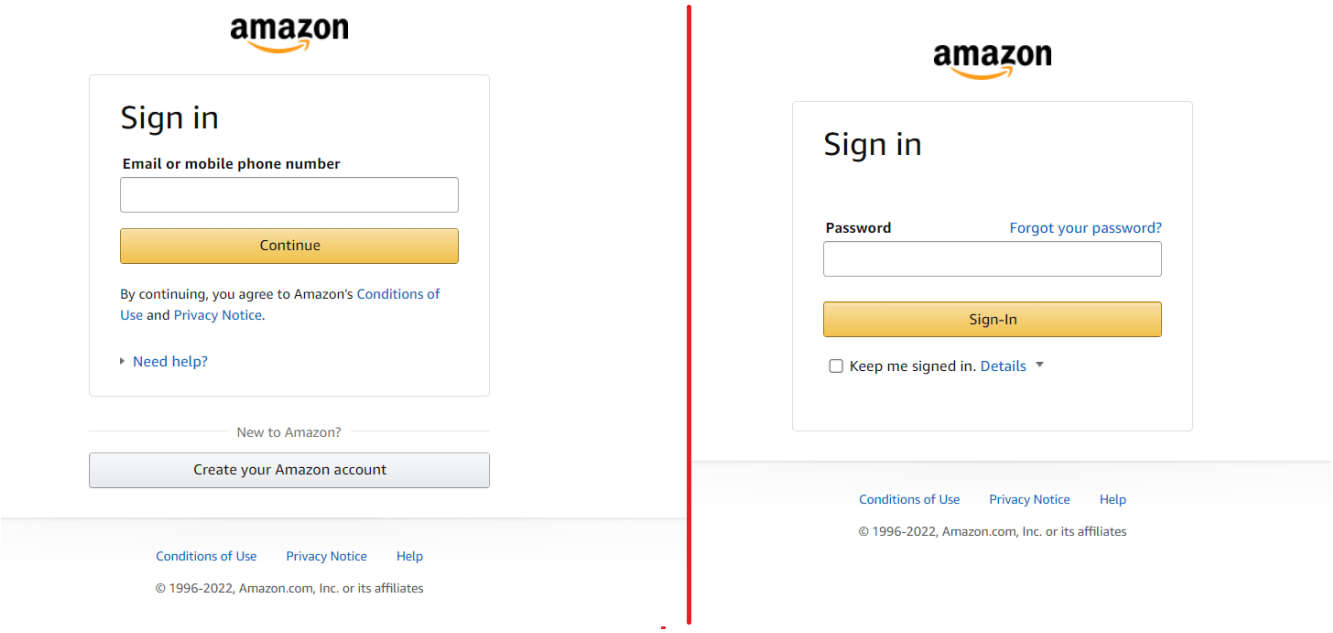
Now when you run this code, Scrapy will login into the **QuotesToScrape** website and then you can start scraping as if you were logged in.

Login Method #3: Using Headless Browser To Login

The above example will work for a lot of simple real-world websites that you will find today. However, it is still too simplistic compared to the login process of most modern websites.

So to give us a more realistic example, we're going to look at how to log into Amazon.com and then scrape product pages whilst logged in.

Amazon uses a 2-step login process with a username/email page and a password page, and as we will see from the Payloads it requires a lot more data to successfully log in.



To get started we will go through the login process like before with the **Network Tab** open.

From looking at the **Payload Tab**, we can see that Amazon sends a lot more data to the server than **QuotesToScrape** during **Step 1 (enter username/email)**:


```
appActionToken: o35S9FuQrkaFvid5z3ZU9fGB1loj3D
appAction: SIGNIN_PWD_COLLECT
subPageType: SignInClaimCollect
openid.return_to: ape:aHR0cHM6Ly93d3cuYW1hem9uLmNvbS9ncC95b3Vyc3RvcmUvaG
prevRID: ape:OEcyQjFIUUJTVloxVDdCQVc3V1E=
workflowState: eyJ6aXAiOiJERUYiLCJlbmMiOiJBMjU2R0NNIiwiYWxnIjoiQTI1NktXI
email: myemail@gmail.com
password:
create: 0
metadata1: ECdITeCs:tj/Z1TGgzan7hZN7Vk4eDMcg3XuvZ2s6PKY9bZweg1RoA7PNkL0g
kWrDFA1z8Zi3gtnT2crJY1tERAfHiogMHHq6viaBakraGDak11UGGbuf1H9sgbFO91O2ho6C
qekYcs863XHDIpbvMl3fINwrT8P1bJ33W8lAylI6jDK6Hf4oxgby4vfIf7WzcPyWfp6qiqlF
buC9SEesPWuSlVGHvsxLneL8tATs+QgLIspU/xkZSsmLocB6YGbDv3QyrL9vfI2FsFTYMBM5
Y2977zg0p1OpvuogCahYGAXKXtXQag61oquasblL5GQKMgZDZHFeVO8DJ+Nf+k2OdIRFIeDz
FHMBvk7hIpeaKpbNA5tCnnUE5znOuDeqEmdfUEdjkdnvatLVutb39cwBQMfz2wcr97OR9CHZ
vwf9CjN7RUIbiNlPjk4+UDSTiKzR0BZlW0qF7G/5eYSbyB3CDt7+XBoavRkSSDmDuXi40uLz
wjWFypvFAwnTkb2GhZ4E82P3BDY6r+82RiMFbiPXRhhTau4L/Kbg0pU6Gf/GtxOFpqFur2tL
Fb6c+GVlfgAwYAvrxCUud8UW5b8TDHQyNyngQIP78w+n/N9dwVjkeJli2DtJjSXImX0llCE+
FjNtF/9FX+XIKEPvNnMNyCG8K5XrZFKrZi5mAYnS6SjLfEl21380OEUB+1CGEJBVqIKUNjRH
KChvZp1heS7Sgr9JIz32sksiWXC4fvPs9Jk9FS5JjIZJOIz1ujaT/33wjTObhu+bXE1HRP/O
DkQYN67urwKAQzCS3UfIesrRGEGfiGK95yttCPo0dXqGH6doqkJnoorppE6KLKwmNUvH1MS6
nxmFGyZvxTEOHX45hNeVPhmXQE3eNybGldlBGRSRsiTYjPOCcijhLMxSO5Xyu50fxD2mz/8I
Rd134KS9sUGRW08HrLCWS+gxzeBbllsmY/3nwHs3yj7Hafm40zcmEmAtEeRNvnjmiLdaLwRk
G2px45Dix1yUJWU/98QwJBQ1kidjE6B5GqOhBGTPphvPQS0g8eUWDMhEu49LWC+2oa3ElNZi
vmGyHK4oHV3DUFykVQPZkAMYovjAeoFW0ixCKideSTIBNVR8xADOi0Blry9QIVGAeqFmA3r4
icsHcKLwF0RmEwKgwb6W5qM6NlijIPZakD5g5thf4ZCj0kkjYZCI1/C1SYN6pHFI7XcUnVWK
V1PzF05owgZv2XlDIoGtCvKXEagTU2mSoMmthrIJiir8bup6GwamVOcqI00Ee28D9DrJ964o
lWl0PAy8noGrPUBSOdb99agqQKBSHZYvA7rSHhzDLJqEstsN7f3EwhmgoqMVKEe/OiXnDjfm
e9jI9PvzodoZuRpWpiYG1QgDSZVnMMRgkehCpjlt11lm9df5TE3wDZokJN9J7h4tczQns1Ye
aaToken: {"uniqueValidationId":"c7566eee-84f5-4e9a-8178-1bd9de6a1ff6"}
```


Most of this extra data is very easy to get as `appActionToken`, `appAction`, `subPageType`, `openid.return_to`, `ape`, `prevRID` and `workflowState`, are all sent in the initial HTML response by the Amazon server and stored in hidden `<input>` fields.

However, the `metadata1` and `aaToken` values are actually generated on the client side so they aren't available in the initial HTML response.

Secondly, when you go through **Step 2** of the login process (enter password), you will see even more data is added to the login payload.


```
appActionToken: o35S9FuQrkaFvid5z3ZU9fGB1loj3D
appAction: SIGNIN_PWD_COLLECT
metadata1: ECdITeCs:py9Beb4zBnanaB40FHo1SXuwSkRkdQFh4boLE8kOJxSPSZPKULOp/
VsdxIyyKrVmyiJbkfU1mYIyy2Ru4TcgiP+jUILdQUnIRo0XubtFMMhxj73nK5DQUSR7a5u8dH
DRqkWSYrPjMp4GCd8vwQg6xzrY/SnZVOGrwfJvVh3Zxg4UaOIScVMyEzkWyJ1GeT7o9XH+zsw
nbixk1NEzkHEokjIsIQFn4gZbJU77gShbJQFCmLpiYMbP5eQMsPJ7NX6UIZr8mQ3mQJQeJDEV
LR83xJbMGfsuw4VwVBQdojkamx+ljptUnjCWyFrtw9A8WLZDA8M0uu1Vjnu4yKjzl1lZAmLPr
u5GDHrFlZSRg43AMaqMNFqJ8LhjKayOT97gelUbQiNo9ze0XTrTmnA64y3LosxKV4eWt3ngn4
NMKL2ITwZs3FkpRq3+c5eeUcdUjjdv9VIIiZVHl3FtKcs+vbe4RS3vbC2ayti6dQgSALxW9yI
HpNvEJUeadcdxYAaNRseaffeLvIdhwYzqnTYMGN55J6IVD2LztO5njH1jJ7za34/eTK8hdmdr
XPQ7JYzhQgbOF5nr+hHE3C4dZzbVlJw77sEbmq9ejRhJBPLXlwDQcjLJXNKMGIKHmbgS+RAaC
a0303+v3Zwp4uyISLFCRz48oejpi4CSmldcvYE/1de5W9j+fselNn7PendseDvbYQ/CLeIvvg
imm0NspweJ0qASHYMNoLPBolxeeUqPGnBdHwI6dcKxO8DFmK9vFFqiiuxyOuqNOzDsaJ2WdGU
W2aAnl6ZOhad1aSegRDApUl7Sc3RdbQ8F6O+FI1oDpqblM7QsKwY9qX6sKYGzsdQWcn0lXVF5
Yhy8zJtGml+aBZLI659NPAAPulTo864u1WN3BFvR+DF3dRRPyxqyW+8K8/lQd5WwEyjANA5/r
tsMpWHr/3mgy91ccQdUcw9SypWulK4y3TXhRL+fq4Belfbw+4lVqFDgKGPsGzzM/lYJ4tC3K/
AxY6YpWBFuw0qhgUgcWxt/ipJYtlNF7EZJ2NKzowlUwJfguW8fqJlnL/l2i5hxeKif/jjpYzH
B7vXQJwW9O8lRpaH4zH76SkjalHsZKBQYYr1fHZUrgD30gbQ2XUI4U0G0YKIAMK0SCT/6uBIh
ylJhC32TG6XF31Ck2mVzF9k5u/9r9qQiiFS7N6wNz4q7OEOP+/IKaM/4aNWbcKcVlan8By5r/
EilWLJw8bqJcn+GLXk4p5e+8RYtzhUDRBUB2TJpdZmszLrGRDAlHdzCf+GicrsB6323lhvyrE
gm/9evNV4YRWsVODT7MNLMz2AIHSnKj7d5yRntkb4omlbwwjTPz+MsH2Tf+VkgAoe9Q2Vg9yE
KAEglxY+oy54vvC36C8YAsjteMdZuqGJkY45AxDrCxl5i559dUNw9cbcXGIZAaTn4ku2HawO
lkfNWOj5emTOTiZJ2o48gurKP4Up3i88HXEAQbvrdUga55wWF3IkwwXh3IF+H9XWmXU3WO7x
oTrA4sPkchZsNIgWI+FiKIs2xKvznxhO80x3O7S3Mqsr7MnDbFUXSbri8KB1opkNgKsAs5NU
mviWnL2UBn5RubtMGp5HDg1pvN1sGsTQMMhWyfrdNlqbYIBOp27SqfXwzaKU4awKNpUJ+RSaC
WIDWnDFVZD2SaqEyyH55J6TBCabG3mXnJWR6JbUIQ2Ig2X5Q6HN8kjSsLLMejHy1jRaOB0fq9
hGFAufSU584l1ZnnM2TykGuQurMrjysi/SGDdKXBFAsYt+DcYz+7rIXOT9cF/Trsegcgvad2n
P1VijeEJg8+UHHpgLh+RssrQl19LMQMy8yDn8ISVygd3JYeB9LBWGmFVB3SZCxsnEGx7ArmLC
RHaMflWduaMQtBr2nrX2j6pIZdEWXG4RyO9np11A+Ih7ob3qzq0zb4SAjV7qvwY8q8SYO6BaC
N+0GkyDiueSAhjbglF6L9JaVe2z2mWjPVjkqYlBmcZ0cYCA5ujuMuAu7fvhHakiDdUFS0ZnsC
+frjjpJWafwR4gh7VfMG+xxfMf0C/3M83Dy/q1/u9Xy9BUmihXPQEI1kc886X1Scr4/gqMgmD
muldHfWZnplISmB2A94zHWjcNnDvVpqvLKO1N0EQhNPbiVkJjFCps9xpDMeI384qqMBG6//4c
r23FKvza9DsACOIqTabEE1rFqdFtLj++PhtgdHwNJSLpoeQLzTSFlKlJVJWWt1LRXxXV1XWqk
openid.return_to: ape:aHR0cHM6Ly93d3cuYW1hem9uLmNvbS9ncC95b3Vyc3RvcmUvaG9
prevRID: ape:NzFGSFIzV0hCRVlXQjVRSjdUMjg=
workflowState: eyJ6aXAiOiJERUYiLCJlbmMiOiJBMjU2R0NNIiwiYWxnIjoiQTI1NktXIn
YPxaLVGzyoipuuyZT76sgI_zH4ji8c94ACO6IWyKTzfAz9uXaVNJkj_athErc3fbg6pejO0Ge
wwBbgARfsvfAZDTXcBOCt2ce6CDN3R07GBShKS627Ex-2D2_B8.7ZnRgS-szSdtj6FIS6W5bg
```

```
email: myemail@gmail.com
email: myemail@gmail.com
encryptedPwd: AYAAFK1kFz5NS47OZNL1Eeq7gM8AAAABAAZzaTptZDUAIDU2ZDE0ZWRjZTh
w7ZZ+MWIUF9kP3nZrn2UmY1F6Cj0MCOnGTUQk9IaLHW4V+lZ5M5tr7zyTqqvjz/kgdms4vPMl
encryptedPasswordExpected:
aaToken: {"uniqueValidationId":"ccfd1cad-66da-4195-9f9e-41c6ef794d67"}
```

Here a lot of things have changed:

- The `prevRID` and `workflowState` have recieved new values from the server
- The `metadata1` and `aaToken` values have been generated on the client side again
- The password is encrypted in the client and stored in `encryptedPwd`.

Because Amazon is generating the `metadata1`, `aaToken` and `encryptedPwd` on the client side which our normal Scrapy requests can't replicate, we are faced with two options. Either:

- **Option 1:** Reverse engineer the JS code Amazon is using to generate the `metadata1`, `aaToken` and `encryptedPwd` values on the client side, or,
- **Option 2:** Use a headless browser to login, and leave it generate the `metadata1`, `aaToken` and `encryptedPwd` values for us. Then extract the session cookies and use them with our normal Scrapy requests.

Option 1 could be very time consuming to implement and unreliable over the longterm, so the best and easiest option is to go with **Option 2**. Use a headless browser for the login process and then continue with normal Scrapy requests after being logged in.

You could use any headless browser Scrapy integration for this, however, for this example I'm going to use Scrapy Splash as it integrates well with Scrapy.

> **SCRAPY SPLASH**
>
> I'm not going to explain too much how Scrapy Splash works, but if you are new to Splash then check out our Scrapy Splash guide here if you want to learn how to setup and use Splash.

In this script we will use our Scrapy Splash headless browser to:

1. Go to Amazon's login page
2. Enter our email address, and click **Continue**
3. Enter our password, and click **Login**
4. Once logged in, extract the session cookies from Scrapy Splash
5. Start scraping the pages we want to scrape, by making normal Scrapy requests and adding the session cookies to each request.

```
import scrapy
from scrapy_splash import SplashRequest

lua_script = """
function main(splash, args)
    splash:init_cookies(splash.args.cookies)

    assert(splash:go(args.url))
    assert(splash:wait(1))

    splash:set_viewport_full()

    local email_input = splash:select('input[name=email]')
    email_input:send_text("EMAIL@GMAIL.COM")
    assert(splash:wait(1))

    local email_submit = splash:select('input[id=continue]')
    email_submit:click()
    assert(splash:wait(3))

    local password_input = splash:select('input[name=password]')
    password_input:send_text("PASSWORD")
    assert(splash:wait(1))

    local password_submit = splash:select('input[id=signInSubmit]')
    password_submit:click()
    assert(splash:wait(3))

    return {
        html=splash:html(),
        url = splash:url(),
        cookies = splash:get_cookies(),
        }
    end
"""
```

```
class AmazonLoginSpider(scrapy.Spider):
    name = "amazon_login"

    def start_requests(self):
        signin_url = 'https://www.amazon.com/ap/signin?
openid.pape.max_auth_age=0&openid.return_to=https%3A%2F%2Fwww.amazon.com%
        yield SplashRequest(
            url=signin_url,
            callback=self.start_scrapping,
            endpoint='execute',
            args={
                'width': 1000,
                'lua_source': lua_script,
                'ua': "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
                },
            )

    def start_scrapping(self,response):
        cookies_dict = {cookie['name']: cookie['value'] for cookie in res
        url_list = ['https://www.amazon.com/']
        for url in url_list:
            yield scrapy.Request(url=url, cookies=cookies_dict, callback=

    def parse(self, response):
        with open('response.html', 'wb') as f:
            f.write(response.body)
```

To use this code, we also need to update our Scrapy projects `settings.py` file to activate Scrapy Splash.

```
# settings.py

# Splash Server Endpoint
SPLASH_URL = 'http://localhost:8050'


# Enable Splash downloader middleware and change
HttpCompressionMiddleware priority
DOWNLOADER_MIDDLEWARES = {
    'scrapy_splash.SplashCookiesMiddleware': 723,
    'scrapy_splash.SplashMiddleware': 725,

'scrapy.downloadermiddlewares.httpcompression.HttpCompressionMiddleware':
810,
}
```

```
# Enable Splash Deduplicate Args Filter
SPIDER_MIDDLEWARES = {
    'scrapy_splash.SplashDeduplicateArgsMiddleware': 100,
}

# Define the Splash DupeFilter
DUPEFILTER_CLASS = 'scrapy_splash.SplashAwareDupeFilter'
```

Using this headless browser method we can bypass the complexities of trying to reverse engineer a websites login process and just let the headless browser do the hard work.

Not Getting Blocked After Logging In

So we've covered how to actually login into a website and scrape data behind the login. However, that isn't the end of the problem.

When scraping whilst logged into a website, it is much easier for the website to identify you as a scraper as every request you make can be pinned back to your account.

Not only is it easier for the website to detect you as a scraper, the consequences of getting caught are a lot higher too as the website might have personal information about you (name, email, credit card info, etc.) and you have explicitedly agreed to their terms & conditions. More on this later.

As a result, you should always take extra steps to ensure your scrapers don't get detected when scraping whilst logged in.

Here is a list of steps you should consider taking:

- Use Multiple Accounts
- Static IP Addresses
- Browser Profiles & User-Agents
- Use A Single Thread
- Delays Between Requests
- Realistic Request Patterns

Depending on the sophistication of a websites anti-bot countermeasures, all of these mightn't be necessary. However, in all cases they will reduce your chances of getting blocked.

Use Multiple Accounts

This isn't always possible (if you need paid accounts or KYC for each account), however, if it is possible you should use multiple different accounts when scraping behind a login.

If you can spread your scraping across tens or hundreds of accounts then it is much easier for you to scrape undetected as you can ensure you never send a unrealistic number of requests through a single account.

Static IP Addresses

You should use the same static IP address for each account you use to scrape a website behind the login.

If your IP address changes with each request, or everytime you login you use a different IP address then it can make it easier for the website to see that you are a scraper. Especially, if you use a IP address in a different location with every request.

Also, to increase security, a lot of websites are now requiring you to validate your identify with a text message, email, or authenticator app everytime they detect you logging in using a new IP address.

As a result, if you aren't using a static IP address with each account then you will have to build a entire system to validate these security prompts everytime you login which just adds unnecessary complexity to your scraping.

Residential IPs are better than datacenter IPs, but depending on the website a static datacenter proxy might work just fine.

Browser Profiles & User-Agents

By default, Scrapy sends the following user-agent with every request:

```
user-agent: Scrapy/VERSION (+https://scrapy.org)
```

This user-agent clearly identifies your scraper as bot, so it is highly likely to get blocked.

As a result, you should make sure that you are using realistic user-agents (better yet full browser profiles) when sending your requests, and that the user-agent should remain constant for the entirety of the scrape once logged in.

For more information about setting fake user-agents in Scrapy then check out our user-agent guide here.

If you should like to get an up to date user-agent or browser profile then check out our free Fake Headers API.

Use A Single Thread

A real user is highly unlikely to be making more than 1 page request at a time to a websites servers.

So to decrease the chances of your scrapers being detected you should set your CONCURRENT_REQUESTS to 1 in your `settings.py` file or in the scraper itself.

```
## settings.py

CONCURRENT_REQUESTS = 1
```

Delays Between Requests

When a human browses a website, they take their time and can spend anywhere from 1 to 120 seconds on each page. However, if your scraper sends requests one

after another with no delay between requests then this is a clear sign that you are in fact a scraper.

Therefore, you should use Scrapy's DOWNLOAD_DELAY. In your `settings.py` file or in the scraper itself you should set your `DOWNLOAD_DELAY` to at least **10** which will use a random delay of between 5 and 15 seconds with each request.

```
## settings.py

DOWNLOAD_DELAY = 10
```

For more info, on **DOWNLOAD_DELAYS** you can check out our guide here.

Realistic Request Patterns

Another way that your scraper could be detected is if you are requests URLs in a order/pattern that a normal user would never do.

For example, a normal user once logged into a e-commerce store for example would search a product say 'iPads' then click the product links that appeared in the search results.

However, if your scraper never visits the search page and uses slimmed down URLs that the website normally doesn't show to users then a website might detect this and ban your account.

Risks Of Scraping Behind Logins

Compared to scraping publically available web pages, scraping behind logins carries with it a lot more risks for you as a developer and company:

Risk 1: Personal Information

When you created the account, you most likely had to give the website personal information (name, email, telephone, credit card, etc.) that the website can tie back to your scraper.

Risk 2: Account Bans

Another risk, is if the website is detects your scraper then they could block/delete your account and ban you from creating any new accounts.

Sometimes, losing an account mightn't be a big deal. However, if your Facebook, Instagram, etc. account got banned then you could lose all your photos, messages, Facebook Ads account and never be able to create another account again.

Or if you were using a paid account, then you could be locked out of your account with no refund.

Risk 3: Lawsuits

Scraping public web pages is a bit of legal grey area, however, when it comes to scraping behind a websites login then the law and legal precendants are much clearer.

When you create an account with a website, you explicitedly agree to a websites Terms & Conditions which can forbid the scraping of their content. So if you create an account with a website than forbids web scraping and you scrape their website anyway then you do open yourself up to potential lawsuits.

Nearly all successful web scraping lawsuits have been when a user scraped behind a login, and some websites can be very aggresive enforcing this and can look to make an example out of users caught scraping behind logins.

So before scraping behind a websites login, you should first review their Terms & Conditions and then make an informed decision on whether the data you would obtain from scraping behind a login is worth the legal risks involved.

More Scrapy Tutorials

So theres an overview on how to login to websites using Scrapy and scrape non-public data.

If you would like to learn more about Scrapy, then be sure to check out The Scrapy Playbook.